CS102L Data Structures & Algorithms
Lab 1

Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Spring 2026

Contents

Introduction

1.1 Instructions

- This lab will contribute 1% towards your final grade.
- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *lab1_aa12345.zip* where *aa12345* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 100.

- 50 Marks are for the completion of the lab.
- 50 Marks are for progress and attendance during the lab.

1.3 Lab Objectives

Following are the lab objectives of this lab:

- Understand the fundamentals of Python Tuples.

1.4 Late submission policy

There is no late submission policy for the lab.

1.5 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.6 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
    assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, "x should be a non-negative number"
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
    pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
    pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
    pytest <filename>.py
```

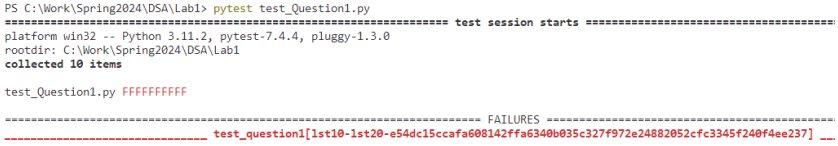
If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================= test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

============================== 10 passed in 0.06s =========
```

However, if all tests fail, then you should receive a similar output.



Lab Exercises

3.1 Last name first

This function should be implemented in the file **q1.py**.

3.1.1 Challenge

Write a function named `last_name_first` that accepts a single parameter t, which is passed as a list of tuples. Each tuple contains a name in parts *(first name, middle name, last name)*. Your function should modify each name so that the last name appears first in the tuple.

3.1.2 Note

This function modifies a list in place and, as such, should not return any useful value.

3.1.3 Sample Interaction

```
>> t = [('Ahmed', 'Dawood'), ('Haroon', 'Hussain', 'Fawad',
   'Rasheed'), ('Muhammad', 'Faisal', 'Amin')]
>> last_name_first(t)
>> t
[('Dawood', 'Ahmed'), ('Rasheed', 'Haroon', 'Hussain', '
   Fawad'), ('Amin', 'Muhammad', 'Faisal')]
```

3.1.4 Constraints

t is a list of tuples, where each tuple has one or more strings in it.

3.1.5 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q1.py
```

3.2 Dude, Where's My Robot?

This function should be implemented in the file **q2.py**.

3.2.1 Problem

Your neighbor's garden robot was hacked and moved from its charging pod where you had last left it. They cannot find the robot and need the grass trimmed right now for a party with their committee friends this evening. Luckily, they know of your skill with computers. You manage to retrieve the last set of instructions sent to the robot. Now you just have to figure out how far away the robot has moved. The set contains n instructions. Each instruction is of the form direction distance. On receiving this instruction, the robot moves distance in direction. The robot executes the instructions one after the other. Given the list of instructions, you want to find out the distance that the robot has moved.

3.2.2 Function Description

Write a function `total_distance` which takes the list of tuples movements as a parameter and returns the total distance that the robot has moved from its original distance once it has carried out the given instructions.

3.2.3 Input

The input is a list of tuples where each tuple contains the direction (where direction is one of *UP, DOWN, RIGHT, and LEFT*) and distance of type integer.

3.2.4 Output

Return the total distance that the robot has moved from its original distance once it has carried out the given instructions. Round the distance up to the nearest integer.

3.2.5 Sample Interaction

```
>> total_distance([('UP', '5'), ('DOWN', '3'), ('LEFT', '3')
   , ('RIGHT', '2')])
3

>> total_distance([('RIGHT', 1), ('DOWN', 0), ('LEFT', -2),
   ('DOWN', -4)])
5

>> total_distance([[('UP', 0)])
0
```

3.2.6 Sample Interaction Explanation

In the first sample, the robot moved 5 up, then 3 down, then 3 left, and then 2 right. Reluctantly, the robot is 2 up and 1 left from its original position. The distance is $math.sqrt(2 * 2 + 1 * 1)$ which, rounded up, is 3.
In the second sample, the robot is 3 right and 4 up from its position. The distance is $math.sqrt(3 * 3 + 4 * 4)$ which, rounded up, is 5.
In the third case, the robot has not moved. The distance is 0.

3.2.7 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q2.py
```

3.3 Loan Repayment Strategy

This function should be implemented in the file **q3.py**.

3.3.1 Problem

On the advice of your relative from the stock market, you have invested in stock in the hope to eventually pay off your Habib loan. Your relative sends you daily updates on your stocks in the following form.

Purchase Date	Purchase Price	shares	symbol	current price
26 Aug 2019	43.50	100	HU	47.02
27 Aug 2019	22.07	500	PTI	19.11
30 Oct 2019	51.98	200	JHR	50.14
28 Nov 2019	137.92	50	WTF	150.28

You want to find out your current earnings from this information. In the table above, each share of HU is at a profit of $47.02 - 43.50 = 3.52$. As you have 100 shares of HU, your profit is $100 * 3.52 = 352$.
Similarly, with PTI you are at a loss of $500 * (22.07 - 19.11) = 1480$. Your JHR stocks are at a loss of $200 * (51.98 - 50.14) = 268$ and and your WTF stocks are at a profit of $50 * (150.28 - 137.92) = 618$. Your total profit is $352 - 1480 - 268 + 618 = -778$.

3.3.2 Function Description

Implement the function `compute_profit` that takes stocks information as a parameter in the form of a list of tuples and returns the total profit. Each tuple contains the following items in the given order. The type of each item is shown.

- purchase_date : *int*
- purchase_price : *float*
- shares : *int*
- symbol : *str*
- current_price : *float*

3.3.3 Constraints

- The argument contains at least 1 tuple.
- All tuples in the list follow the format described above

3.3.4 Sample Interaction

```
>> compute_profit([('25-Jan-2001', 43.5, 25, 'CAT', 92.45),
   ('25-Jan-2001', 42.8, 50, 'DD', 51.19), ('25-Jan-2001',
   42.1, 75, 'EK', 34.87), ('25-Jan-2001', 37.58, 100, 'GM',
    37.58)])
1101
```

3.3.5 Sample Interaction Explanation

The input contains information of 4 stocks. The name and corresponding profit from each are as follows.

- CAT : 25 * (92.45 - 43.5) = 1223.75
- DD : 50 * (51.19 - 42.8) = 419.5
- EK : 75 * (34.87 - 42.1) = -542.25
- GM : 100 * (37.58 - 37.58) = 0

The total profit is therefore 1223.75 + 419.5 - 542.5 + 0 = 1101

3.3.6 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q3.py
```

3.4 Social Network User Removal

This function should be implemented in the file **q4.py**.

3.4.1 Problem

You are building a social networking application focused on connecting people based on common interests. The application stores user data in a dictionary users_data where each user's name is the key. The corresponding values include the user's phone number and a list of friends they have connected with. Your task is to create a function that removes user_name and all their connections from the users_data.

3.4.2 Function Description

Write a python function named `remove_user` that takes a users_data and user_name. If the user_name exists, it removes the given username. This is a void function.

3.4.3 Sample Interaction

```
>> users_data = {
..     'Alice': ('123-456-7890', ['Bob', 'Charlie']),
..     'Bob': ('987-654-3210', ['Alice', 'David']),
..     'Charlie': ('111-222-3333', ['Alice']),
..     'David': ('555-666-7777', ['Bob'])
}

>> users_data = remove_user(users_data, 'Alice')
>> users_data
{'Bob': ('987-654-3210', ['David']), 'Charlie': ('
   111-222-3333', []), 'David': ('555-666-7777', ['Bob'])}
```

3.4.4 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q4.py
```

3.5 Updating Employee Records

This function should be implemented in the file **q5.py**.

3.5.1 Function Description

Write a function named `update_record`, that takes the following parameters:

1. employee_records – A list of tuples representing employee data. Each tuple contains the following information: (*ID*, *Position*, *Salary*, *Experience*).
2. ID – An ID of a employee whose data has to be updated.

3. record_title – The type of data that needs to be updated. It can be *ID*, *Position*, *Salary* or *Experience*.

4. data – data (string or int): The new data that should replace the existing data in the specified record_title.

The function should do the following:

1. If record_title is *ID*, return a message: *ID cannot be updated*.

2. If the provided ID is not found in employee_records, return the message: *Record not found*.

3. If the ID is found, update the corresponding record_title with the new data and return a message: *Record updated*.

3.5.2 Sample Interaction

```
>> employee_records = [
..     ("E001", "Manager", 80000, 5),
..     ("E002", "Developer", 60000, 2),
..     ("E003", "Analyst", 50000, 1),
..     ("E004", "Designer", 70000, 3)
]

>> update_record(employee_records, "E001", "ID", "E005")
ID cannot be updated

>> update_record(employee_records, "E001", "Position", "
   Senior Manager")
[("E001", "Senior Manager", 80000, 5), ("E002", "Developer",
    60000, 2), ("E003", "Analyst", 50000, 1), ("E004", "
   Designer", 70000, 3)]

>> update_record(employee_records, 'E005', "Salary", 55000)
Record not found
```

3.5.3 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q5.py
```